

A25

PROJET AKROPOLIS

- LO21 -

Rapport Final

ANCELY Louise
BLIARD Samuel
GENGEMBRE Gabriel
LAVALLEE Antoine
PIERRE Romain

Sommaire

Introduction	1
1. Synthèse	2
a. Console	2
b. Qt Creator	2
2. Description de l'architecture	4
3. Adaptabilité de l'architecture	7
4. Description du planning effectif	8
5. Contribution des membres	13
6. Temps de travail individuel	14
7. Remarque global sur le projet	15
8. Conclusion	16
Annexes	17
Code de génération UML	17

Introduction

Ce rapport rend compte de la réalisation en C++ du jeu vidéo Akropolis, issu du jeu de société du même nom créé par Jules Messaud et paru en 2022. La réalisation de ce projet, mobilisant un groupe de 5 étudiants, s'est déroulée sur un semestre et permet de mettre en application les connaissances acquises dans le cadre de l'UV LO21.

Akropolis est un jeu de stratégie où les joueurs incarnent des architectes de la Grèce antique concurrents pour bâtir la cité la plus prestigieuse en assemblant des tuiles hexagonales comportant divers types de quartiers. Pour l'emporter, il est crucial de gérer ses ressources en pierre afin d'acquérir les meilleures tuiles et de construire en hauteur, ce qui permet de multiplier la valeur des quartiers respectant des conditions spécifiques.

Au-delà de la simple transposition des règles du jeu, ce projet avait pour objectif principal la conception d'une architecture logicielle robuste, respectant les paradigmes de la programmation orientée objet. Le développement a nécessité la modélisation de mécanismes complexes, tels que la gestion d'un plateau hexagonal pouvant s'étendre en hauteur (la 3D étant au cœur du gameplay) ou encore l'algorithme de calcul de scores dynamiques basés sur des motifs d'adjacence

Ce document s'articule autour de quatre axes majeurs reflétant notre démarche de conception et de développement. Dans un premier temps, nous dresserons une synthèse fonctionnelle de l'application, en distinguant les fonctionnalités pleinement opérationnelles de celles qui n'ont pu être abouties. Nous détaillerons ensuite l'architecture technique retenue, en justifiant nos choix de conception à l'aide de diagrammes UML, avant de démontrer la capacité d'évolution de notre solution, notamment sa prédisposition à intégrer de nouvelles variantes de jeu. Enfin, le rapport se conclura sur la gestion du projet, présentant le planning effectif suivi durant le semestre ainsi que la répartition des contributions et la charge de travail de chaque membre de l'équipe.

1. Synthèse

a. Console

Dans un premier temps, nous avons choisi d'implémenter un mode console que nous avons perfectionné au cours du projet. Cette étape était importante car, elle nous a permis de valider la robustesse du code déjà écrit, nous avons aussi pu voir que notre logique fonctionnait correctement. L'objectif de ce mode est d'offrir une expérience de jeu complète en permettant aux joueurs de suivre de manière la plus lisible possible le cours de leur partie, sans forcément dépendre d'un logiciel qui permet une interface graphique comme Qt Creator.

Lors du lancement du mode console, l'utilisateur accède au menu principal lui proposant trois options : créer une nouvelle partie, charger une partie déjà enregistrée ou quitter le jeu. Si l'utilisateur souhaite charger une partie mais qu'aucune sauvegarde n'est disponible, il verra un message d'erreur s'afficher et sera redirigé vers le menu principal. Dans le cas contraire, il pourra accéder à la liste des parties déjà enregistrées pour sélectionner celle qu'il désire. Si l'utilisateur souhaite créer une nouvelle partie, il lui sera demandé une série de questions pour le configurations :

- Mode de jeu : solo / multijoueur
- Le nom de chacun des joueurs
- Si mode solo, le niveau de l'illustre constructeur : Facile / Moyen/ Difficile
- Les variantes qui permettent la sélection des règles optionnelles, accompagnée de leurs descriptions pour garantir la bonne compréhension de l'utilisateur

Une fois la partie configurée et lancée, le déroulement d'un tour de jeu suit une logique précise. Lorsque c'est son tour, le joueur est invité à choisir une tuile parmi celles disponibles dans la pioche. Pour faciliter la tâche au joueur, son score et son nombre de pierres seront affichés. Une fois une tuile est choisi processus de placement se déroule en plusieurs étapes pour garantir la validité des coups :

1. Parmi les 6 rotations possibles, le joueur choisit une orientation. Une visualisation console lui est présentée pour une meilleure visualisation.
2. Le programme va ensuite proposer à l'utilisateur la liste des emplacements valides selon les règles de placement.
3. Si le placement recouvre des carrières, les pierres gagnées seront ajoutées automatiquement à celles du joueur.

Aussi, à la fin de chaque tour, une option permet de sauvegarder l'état de la partie pour une reprise ultérieure. Cette fonctionnalité génère un fichier de sauvegarde, facilitant la persistance des données et la gestion des sessions de jeu.

b. Qt Creator

Une fois que le jeu a commencé à prendre forme en mode console, nous avons commencé à l'implémenter dans Qt Creator. Cela nous permet de déployer une interface graphique offrant une meilleure lisibilité des parties et de les rendre bien plus attractives. Pour ce faire, les utilisateurs ont accès à un grand nombre d'opérations.

Tout d'abord, dès l'ouverture de l'interface, l'utilisateur fait face à un menu de configuration permettant de paramétrer le mode de jeu (solo ou multijoueur), que ce soit en affrontant l'ordinateur via différents niveaux de difficulté (facile, moyen ou difficile) en mode solo, ou en mode multijoueur (2 à 4 joueurs) où chaque utilisateur peut être nommé. Une fois la partie lancée, le joueur peut explorer sa cité en utilisant la molette de la souris pour zoomer et dézoomer, garantissant une visibilité optimale sur ses constructions.

Une fois la partie lancée, le joueur sélectionne ses pièces directement dans la rivière par un clic, une action soumise à la vérification automatique du nombre de pierres disponibles. L'interface offre ensuite une phase de prévisualisation cruciale : avant tout engagement définitif, il est possible de manipuler la tuile, d'observer sa rotation en temps réel et de tester son positionnement sur la grille de la cité. Nous avons fait le choix de ne pas bloquer les rotations même quand celles-ci sont illégales ; cependant, il n'est pas possible pour l'utilisateur de valider le positionnement, ce qui nous a semblé préférable pour l'expérience joueur. Sur le côté droit de l'écran, l'utilisateur peut retrouver :

- Une barre informant sur la complétion de la partie
- L'état général du tour
- Le nombre de pierres du joueur
- Son nombre de points
- Les tuiles disponibles dans la pioche ainsi que leur coût
- Un bouton « Annuler » permettant de sélectionner une tuile à tout moment
- Un bouton « Valider » qui verrouille le placement de la tuile
- Un bouton « Pivoter » qui fait pivoter la tuile lorsque celle-ci est placée mais pas encore verrouillée
- Le bouton « Changer Pivot » qui modifie l'hexagone autour duquel la tuile pivote afin de s'assurer que le joueur puisse placer sa tuile de la manière désirée
- Un bouton qui permet de quitter la partie et de la nommer afin que celle-ci soit enregistrée si le joueur souhaite la reprendre plus tard.

Enfin, la partie se clôture de manière formelle par une fenêtre de résultats synthétique, listant les scores de l'ensemble des joueurs pour désigner le vainqueur.

2. Description de l'architecture

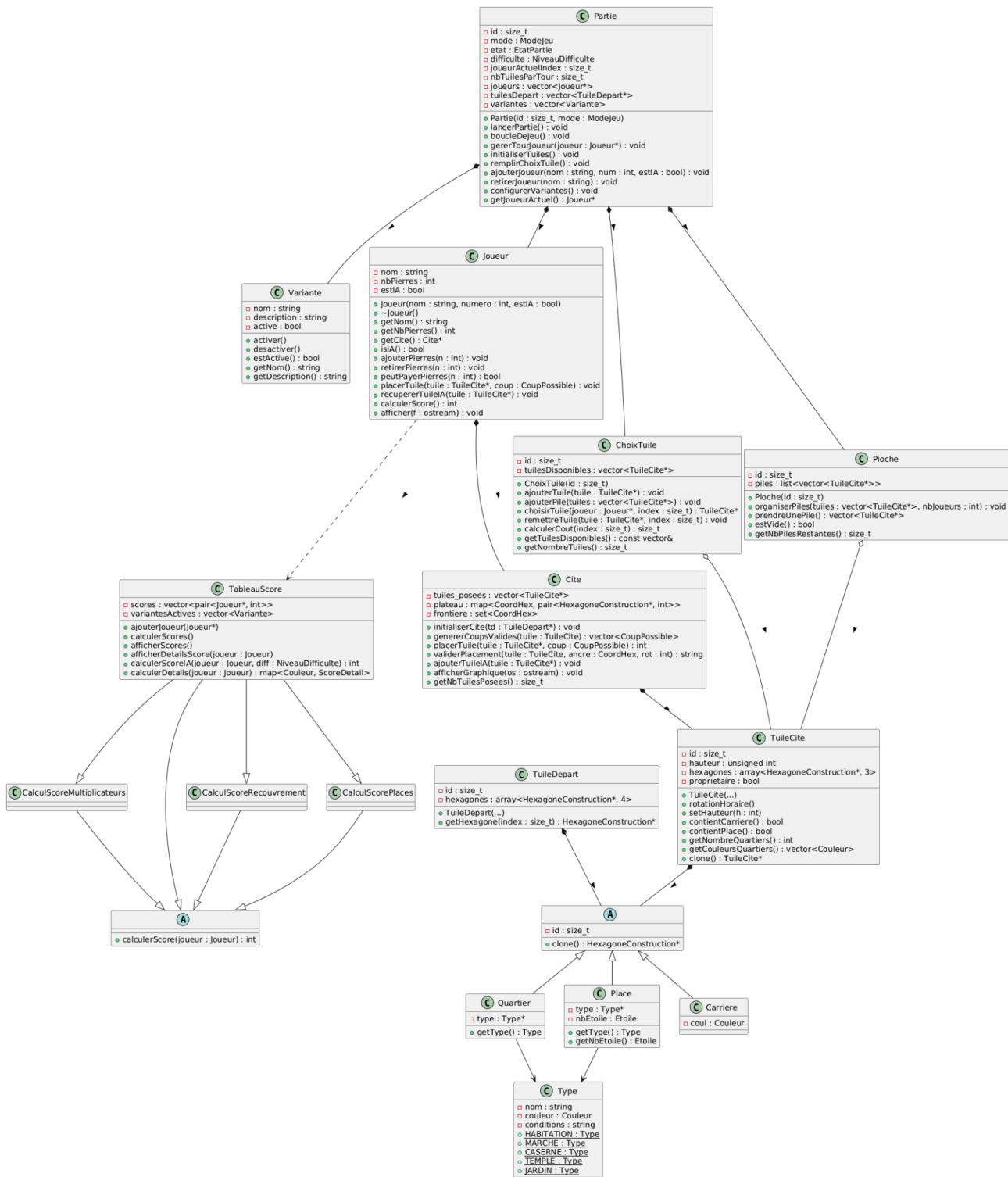


Figure 1 : Modélisation UML de l'architecture du jeu

L'architecture globale du projet est centralisée autour de la classe `Partie`, qui agit en tant que contrôleur principal de l'application. Ce choix de conception est directement dicté par les règles de fin de partie d'Akropolis, qui stipulent que le jeu s'arrête lorsqu'il ne reste plus qu'une seule tuile dans le chantier et que toutes les piles de la réserve sont épuisées. Pour surveiller ces conditions efficacement, la classe `Partie` détient et gère les instances de `Pioche` et de `ChoixTuile` (le marché). Concernant la structure de données de la `Pioche`, nous avons opté pour une `std::list<std::vector<TuileCite*>>`. Ce choix de conteneur, une liste de vecteurs, est justifié par la mise en place physique du jeu qui impose de constituer des piles distinctes contenant un nombre précis de tuiles (3, 4 ou 5) selon le nombre de joueurs. Cette structure permet de déverser simplement une pile entière (vector) dans le marché lorsque celui-ci se vide, respectant ainsi la gestion de réapprovisionnement décrite dans les règles.

La modélisation de la cité des joueurs est gérée par la classe `Cite`, qui s'appuie sur un conteneur associatif `std::map` dont les clés sont des coordonnées hexagonales axiales définies dans `CoordHex`. L'utilisation d'une map a été privilégiée par rapport à un tableau statique à deux dimensions car la cité peut s'étendre indéfiniment dans n'importe quelle direction. Le système de coordonnées axiales (q, r) facilite les calculs de voisinage nécessaires pour valider les règles de pose, comme l'obligation d'adjacence. Au niveau des tuiles, nous avons implémenté une structure polymorphique où chaque `TuileCite` est composée de trois pointeurs vers une classe abstraite `HexagoneConstruction`. Cette classe mère se décline en trois classes filles : `Quartier`, `Place` et `Carriere`. Ce découpage est intéressant pour l'extensibilité du projet et le respect des règles, car chaque type d'hexagone a un comportement distinct : les carrières génèrent des pierres, les places multiplient les scores par leurs étoiles, et les quartiers rapportent des points selon des conditions spécifiques. Les Quartiers portent un `Type` (couleur) déterminant leurs conditions de points. Cette approche garantit l'extensibilité du code : l'ajout d'un nouveau type d'hexagone ne nécessiterait que la création d'une nouvelle classe dérivée, sans impact sur la structure de la tuile elle-même.

La classe `Joueur` a été conçue pour gérer à la fois les joueurs humains et 'l'illustre constructeur' du mode Solo, grâce à un attribut booléen `estIA`. Selon les règles du mode Solo, l'adversaire virtuel ne construit pas une cité cohérente géométriquement, mais accumule simplement des tuiles pour marquer des points. Pour respecter cette spécificité sans complexifier la classe `Cite`, nous avons ajouté une méthode `recupererTuileIA` dans la classe `Joueur`. Cette méthode permet à l'IA de stocker les tuiles dans son inventaire sans passer par la logique de validation spatiale (adjacence, superposition) requise pour un joueur humain. De plus, le calcul du score de l'IA dépendant du niveau de difficulté choisi (Hippodamos, Métagénès, Callicratès), nous avons isolé cette logique spécifique dans une méthode dédiée `calculerScoreIA` au sein du système de score, garantissant une séparation claire entre la logique de jeu standard et les règles spécifiques au mode solo.

Le calcul des points a été externalisé dans une classe dédiée nommée `TableauScore`, séparant la logique de comptage de la représentation des données. Cette architecture est justifiée par la complexité des algorithmes de score, tels que le parcours en largeur nécessaire pour déterminer la taille du plus grand groupe de quartiers bleus. De plus, cette séparation facilite l'implémentation des variantes optionnelles décrites en fin de règles, comme "Les Architectes" ou "Les Jardiniers". La classe `Partie` maintient une liste de variantes actives, et `TableauScore` consulte cette liste lors du calcul final. Cela permet d'appliquer des modifications dynamiques aux règles de points (par exemple, doubler le score d'un quartier sous condition) sans avoir à modifier le code des classes `Quartier` ou `Cite`, assurant ainsi que l'ajout des variantes en fin de projet n'impacte pas la stabilité du moteur de jeu de base.

3. Adaptabilité de l'architecture

L'architecture a été conçue en suivant le principe de séparation des responsabilités, isolant les données, la logique de jeu et l'interface utilisateur. Cette structure modulaire garantit que l'ajout de nouvelles fonctionnalités a un impact minimal sur le code existant.

Il est possible d'ajouter de nouvelles tuiles Cité, car L'architecture utilise un modèle générique pour les tuiles (TuileCite) composé d'hexagones génériques (HexagoneConstruction), ce qui rend l'ajout de nouvelles configurations de tuiles faciles. La création des tuiles est centralisée dans le fichier `fabrique_tuiles.cpp` grâce à la fonction `creerToutesTuiles`. Cette fonction agit comme une usine. Pour ajouter une nouvelle tuile, il suffit d'instancier un nouvel objet `TuileCite` dans une des fonctions appelées (`creerTuiles2j`, `creerTuiles3j`, `creerTuiles4j`) en lui passant les types d'hexagones souhaités (Carrière, Place, ou Quartier avec leurs attributs).

De plus, nous pouvons implémenter de nouvelles variantes. Dans le constructeur de la classe `Partie` (`Partie_Variante.cpp`), il suffit d'ajouter une ligne pour insérer la nouvelle variante dans le vecteur `variantes` : `variantes.push_back(Variante("NomVariante", "Description de la règle"))`; Et d'implémenter la logique de score associée dans `Score.cpp`. La méthode `TableauScore::calculerDetails` récupère l'état des variantes via `estVarianteActive`. Il suffit d'ajouter une condition pour modifier le calcul du score ou le multiplier selon la nouvelle règle. L'impact sur le reste du programme est minime. L'interface de configuration (`configurerVariantes` dans `Partie`) itère dynamiquement sur le vecteur `variantes` pour afficher le menu de sélection. Ainsi, la nouvelle variante apparaîtra automatiquement dans les menus sans avoir à modifier le code de l'interface.

Enfin, un nouveau niveau de difficulté dans le mode Solo peut être créé en ajoutant une nouvelle valeur dans l'enum class `NiveauDifficulte` dans le fichier `GameExcep_Enums.h` (ex: `NiveauDifficulte::EXPERT`). Puis dans `Score.cpp`, dans `TableauScore::calculerScoreIA` il suffit d'ajouter un cas if pour définir comment ce nouveau niveau influence le score. Le reste de la logique du jeu est totalement indépendante.

L'architecture choisie est ouverte à l'extension. Cela assure une maintenabilité durable et une grande facilité d'évolution.

4. Description du planning effectif

Tâche principales	Durée	Date limite (prévisionnelle)
Architecture Logicielle / Modélisation des classes - Jouer au jeu afin d'identifier les concepts clés comme les tuiles, les hexagones, les quartiers, les joueurs et les places. - L'objectif est de créer un MCD qui sera présent dans le premier livrable. - Créer les structures en C++.	48h 25h	28/09/25
Gestion des relations et contraintes (ex : max 3 quartiers, carrières ou places.) - Écrire le code qui gère les interactions entre les différents éléments du jeu. - Inclut toutes les contraintes et relations mentionnées dans ce rapport.	47h 40h	05/10/25
Création des cartes dans la mémoire - Passer sous forme de code les différents objets de classe que nous souhaitons inclure dans le jeu.	8-10h 6h	12/10/25
Implémentation des méthodes - Écriture du code logique qui permet de vérifier la validité des actions des joueurs et de calculer les points. - Il sera nécessaire d'implémenter les méthodes de scoring pour chaque type de quartier et de place	18-20h 13h	02/11/25
Gestion des joueurs et du tour de jeu - Mettre en œuvre le système de tours, la gestion des joueurs - Créer le mode solo avec un Illustrateur Constructeur qui aura plusieurs niveaux de difficulté. - Le jeu doit se dérouler jusqu'à l'épuisement des tuiles.	15-20h 22h30	16/11/25
Interface utilisateur (menu, liste des options, etc) - Développer une interface qui donne envie aux joueurs. - Faire une interface en mode console et une autre graphique avec le framework Qt - L'interface doit permettre - Le paramétrage d'une partie, - La visualisation de l'état du jeu - La vérification de la validité des actions	45-50h 112h	30/11/25

Tests, à vérifier : - Que toutes les fonctionnalités soient bien implémentées - Le calcul des points - La validation des actions - ...	40h 27h30	14/12/25
Rendu du projet, qui inclut : - Le code source - Une vidéo de présentation - Un rapport détaillé		21/12/25
Sauvegarde et chargement des parties	34h30	

Tout ce qui est écrit en gris est issu du rapport 3, et ce qui est écrit en bleu représente les ajouts et modifications. En barre ce qui était prévu initialement mais qui a changé en réalité.

Architecture Logicielle / Modélisation des classes	Durée prévue	Affectation	Avancement	Durée réelle
Création des fichiers et des classes preset	1h	Gabriel	Fin	1h
Créer la classe Partie	2h	Louise	Fin	2h
Créer la classe Variante	2h	Louise	Fin	2h
Créer Joueur	2h	Romain	Fin	2h
Créer Pioche	2h	Gabriel	Fin	1h30
Créer Tableau des scores	2h	Antoine	Fin	1h
Créer Cité	2h	Samuel	Fin	1h
Créer TuileCite	2h	Louise	Fin	2h
Choisir et implémenter les coordonnées hexagonales pour TuileCite (class CoordHex)	2h	Samuel	Fin	2h
Créer HexagoneConstruction	2h	Romain	Fin	1h30
Créer les classes Quartier, Carrière, Place	2h	Gabriel	Fin	1h30

Créer la classe Type	2h	Louise	Fini	2h
Séparation des classes en plusieurs fichiers	1h30	Gabriel	Fini	1h30
Classe Configuration	1h30	Samuel	Fini	2h
Classe MenuConfiguration	2h	Samuel	Fini	2h

Gestion des relations et des contraintes	Durée	Affectation	Avancement	Durée réelle
Création du plateau, gestion du positionnement des tuiles, gestion de la rotation des tuiles	15h	Samuel Gabriel Romain	Fini	25h
Vérification de la validité d'un placement de tuile	6h	Samuel Gabriel Louise	Fini	8h
Inclure les contraintes de placement des différents quartiers	5h	Louise	Fini	5h
Gérer les carrières	3h	Antoine	Fini	1h
Calculer les points	3h	Gabriel Antoine	Fini	1h

Création des cartes dans la mémoire	Durée	Affectation	Avancement	Durée réelle
Générer les tuiles de jeu	4h	Louise	Fini	3h
Gérer le mélange et la distribution des tuiles	4h	Antoine	Fini	2h
Initialiser la réserve de pierres pour chacun des joueurs	1h	Antoine	Fini	1h

Implémentation des méthodes	Durée	Affectation	Avancement	Durée réelle
Implémenter le système de comptage des points en fonction des contraintes	5h	Antoine	Fini	4h
Implémenter les multiplicateurs avec les places	3h	Antoine	Fini	2h
Implémenter l'utilisation des pierres	4h	Antoine	Fini	~2h30
Intégrer des méthodes de vérification des conditions	5h	Samuel	Fini	~4h30

Gestion des joueurs et des tours de jeu	Durée	Affectation	Avancement	Durée réelle
Gérer la pioche et le choix des tuiles	5h	Romain Gabriel	Fini	4h
Gestion du tours à tours	2h	Gabriel Romain	Fini	2h
Gestion du tours à tours en Qt		Louise	Fini	2h
Gérer le mode solo (contre ordinateur)	10h	Gabriel Louise	Fini	6h
Gérer le mode solo (contre ordinateur) en Qt		Louise	Fini	5h
Implémenter le fait de pouvoir quitter à tout moment et le retour en arrière après le choix d'une tuile	4h	Antoine samuel	Fini	3h30

Interface utilisateur	Durée	Affectation	Avancement	Durée réelle
mode console (priorité) :	total	Romain	Fini	~37h

	35h	et Samuel Gabriel Antoine		
Affichage de la tuile lors de sa sélection - UPDATE	2h	Samuel Gabriel	Fini	2h30 3h30
menu (afficher les options : nouvelle/charger une partie) dans version console	10h	Romain	Fini	3h
menu (afficher les options : nouvelle/charger une partie) en Qt		Louise Romain	Fini	2h30
affichage état du jeu (tuile disponible, plateau du joueur ,score)	25h	Antoine (score) Gabriel Samuel	Fini	25h
mode graphique (Qt)	10-15 h	Louise et Romain	Fini	~10h ~35h
Mode graphique - affichage ASCII pour la console	6h	Gabriel	Fini	6h

Tests	Durée	Affectation	Avancement	Durée réelle
test interne au tours (tours a tours) + Correctifs de certaines règles qui n'étaient pas bonnes (nombre de choix de tuiles par pile)	2h	Gabriel	Fini	4h
test enchaînements des tours	4h	Louise	Fini	3h30
test partie complète (6-7 partie) + Correction des problèmes de placements (détection des coups valides et placement d'une carrière sur une carrière) + Tests et correction IA + Test et correction gestion des pierres	4h	Louise Romain	Fini	~10h
Optimisation global du programme au cours du temps	?	Tout le monde	Fini	10h

Sauvegarde et chargement des parties	Durée	Affectation	Avancement	Durée réelle
Architecture et choix de l'implémentation	4h	samuel	Fini	~4h30
Implémentation en cpp	10h	samuel	100%	25h
Test fonctionnement/limites	3h	samuel	100%	5h

5. Contribution des membres

Lors du début du projet, l'ensemble du groupe s'est focalisé sur la structure et sur le mode console, puis lorsque le mode console commençait à être utilisable, on a scindé le groupe en deux : une partie qui s'est focalisée sur le mode graphique en Qt et le reste qui a continué à développer et embellir le mode console. Pour le mode console, il y a eu notamment beaucoup de changements au niveau de l'affichage.

Nous n'avions pas estimé correctement la complexité et le temps nécessaire à la réalisation de certaines tâches, c'est pour cela que la répartition a parfois varié avec ce qui était initialement prévu, afin d'équilibrer la charge de travail de chacun.

De plus, nous nous sommes adaptés aux impératifs personnels de chacun pour que le projet avance en continu. C'est à dire que si pendant une semaine l'un de nous était occupé, nous prenions le relais sur son travail et ce dernier rattrapait en travaillant plus la semaine d'après.

En effet, vers la fin du projet, les rôles étaient beaucoup moins définis. Pour finir au mieux le projet, chaque personne a participé aux différentes tâches qui restaient à faire, notamment pour optimiser et simplifier le fonctionnement ou même quand un bug était repéré. Durant tout le projet, un groupe messenger était très actif, ce qui était notre moyen de communication principal en plus des discussions lors de réunions, durant lesquelles nous travaillions ensemble.

Lors du semestre, nous sommes passés à se faire des comptes rendus hebdomadaires à journaliers sur l'avancée de notre travail personnel.

En résumé, nous sommes fiers de notre travail en groupe et de notre organisation collective. Chacun a su trouver sa place dans ce projet.

6. Temps de travail individuel

	Temps de travail (en h)	part de contribution (en)	Livrable
Gabriel	~60h	20%	Rapporte 1
Samuel	~60h	20%	Rapport 3
Louise	~60h	20%	Rapport 2
Antoine	~60h	20%	Vidéo
Romain	~60h	20%	Rapport Final

7. Remarque global sur le projet

Dans sa globalité, la réalisation de ce projet s'est particulièrement bien déroulée. La communication entre les membres du groupe a été fluide tout au long du semestre ce qui nous a permis de revoir et de corriger nos attentes en termes de deadline et de restructurer la répartition des tâches pour permettre à chacun d'avancer simultanément sans se bloquer. La charge de travail a été exponentielle, ce qui était prévisible car il était plus simple d'avancer une fois certaines connaissances de TD acquises. Cela justifie la différence de réalisation des tâches entre la date prévisionnel et la date réelle (c.f [Description du planning effectif](#)). Chaque membre a été investi identiquement dans ce projet et une entraide a été mise en place sur les points difficiles de développement. Si un tel projet était à reproduire, l'expérience acquise au cours de celui-ci nous permettrait maintenant d'être plus lucide sur l'ordre et la répartition des tâches à réaliser et sur le temps à y passer, bien qu'il est toujours difficile de respecter un planning à la lettre dans ce genre de réalisation.

8. Conclusion

Cela fait maintenant plusieurs mois que nous avons commencé la réalisation de ce projet, réputé pour être l'un des plus chronophages et formateur de notre parcours de GI à l'UTC. Nous l'avons bien remarqué, ce fut bien plus qu'un simple exercice de programmation dans un langage qui nous était encore inconnu à certains en ce début de semestre. C'était un vrai but de traduire un jeu de société accessible à partir de 8 ans en une application de programmation orientée objet complexe. Ce projet nous a vraiment permis de nous projeter dans les conditions réelles de développement auxquels nous pourrions faire face dans le futur, de la conception abstraite à l'interface finale.

Face à la complexité du des règles d'Akropolis nous avons été obligés de concevoir une architecture logicielle sophistiquée. L'application des principes de la vue en cours n'était plus une option mais une nécessité pour gérer la multitude d'entités du jeu. Cette expérience nous a permis de comprendre comment structurer un projet de cette envergure. Ce projet intégrait aussi l'environnement de développement Qt Creator, inconnu pour chacun d'entre nous au départ. Il nous a été nécessaire d'apprendre à travailler dessus pour proposer à l'utilisateur une interface agréable à jouer, offrant un support graphique avec lequel les joueurs peuvent interagir. Cela nous a permis d'aboutir à une interface lisible dont nous sommes fiers. Pour cela, il a fallu lier le cœur du jeu à une interface visuelle fluide. Notre travail sur Qt a rendu notre projet plus accessible et attractif, comme demandé dans le cahier des charges. C'est une mission à laquelle nous serons amenés dans notre vie professionnelle : rendre accessible informatiquement ce qui est complexe.

La réussite de ce projet repose également sur une collaboration fluide et une organisation rigoureuse entre les membres de l'équipe, rendues possibles par l'utilisation de GitLab. Bien que la gestion des conflits de fusion ait demandé une certaine courbe d'apprentissage, il nous a permis de centraliser le code et de coordonner nos développements respectifs, tout maintenant un historique de code propre. Nous avons alors pu nous familiariser avec les flux de travail professionnels (branches, commits, merge requests), indispensables à tout futur ingénieur.

Pour conclure, ce projet a été une épopée enrichissante qui nous a permis de mettre en pratique les notions vues en cours et sur le plan personnel de mettre en valeurs notre persévérance et capacité à travailler en groupe. La répartition des rôles a permis à chacun d'apporter sa pierre à l'édifice, créant une véritable synergie pour coordonner nos forces. Nous sommes aujourd'hui fiers d'avoir abouti à une application fonctionnelle et complète, témoignant de notre capacité à mener de bout en bout un projet informatique complexe.

Annexes

Code de génération UML

@startuml

```
class Partie {
    -id : size_t
    -mode : ModeJeu
    -etat : EtatPartie
    -difficulte : NiveauDifficulte
    -joueurActuelIndex : size_t
    -nbTuilesParTour : size_t
    -joueurs : vector<Joueur*>
    -tuilesDepart : vector<TuileDepart*>
    -variantes : vector<Variante>
    +Partie(id : size_t, mode : ModeJeu)
    +lancerPartie() : void
    +boucleDeJeu() : void
    +gererTourJoueur(joueur : Joueur*) : void
    +initialiserTuiles() : void
    +remplirChoixTuile() : void
    +ajouterJoueur(nom : string, num : int, estIA : bool) : void
    +retirerJoueur(nom : string) : void
    +configurerVariantes() : void
    +getJoueurActuel() : Joueur*
}
```

```
class Variante {
    -nom : string
    -description : string
    -active : bool
    +activer()
    +desactiver()
    +estActive() : bool
    +getNom() : string
    +getDescription() : string
}
```

```
class Joueur {
    -nom : string
    -nbPierres : int
    -estIA : bool
    +Joueur(nom : string, numero : int, estIA : bool)
```

```

+~Joueur()
+getNom() : string
+getNbPierres() : int
+getCite() : Cite*
+isIA() : bool
+ajouterPierres(n : int) : void
+retirerPierres(n : int) : void
+peutPayerPierres(n : int) : bool
+placerTuile(tuile : TuileCite*, coup : CoupPossible) : void
+recupererTuileIA(tuile : TuileCite*) : void
+calculerScore() : int
+afficher(f : ostream) : void
}

class Cite {
    -tuiles_posees : vector<TuileCite*>
    -plateau : map<CoordHex, pair<HexagoneConstruction*, int>>
    -frontiere : set<CoordHex>
    +initialiserCite(td : TuileDepart*) : void
    +genererCoupsValides(tuile : TuileCite) : vector<CoupPossible>
    +placerTuile(tuile : TuileCite*, coup : CoupPossible) : int
    +validerPlacement(tuile : TuileCite, ancre : CoordHex, rot : int) : string
    +ajouterTuileIA(tuile : TuileCite*) : void
    +afficherGraphique(os : ostream) : void
    +getNbTuilesPosees() : size_t
}

class Pioche {
    -id : size_t
    -piles : list<vector<TuileCite*>>
    +Pioche(id : size_t)
    +organiserPiles(tuiles : vector<TuileCite*>, nbJoueurs : int) : void
    +prendreUnePile() : vector<TuileCite*>
    +estVide() : bool
    +getNbPilesRestantes() : size_t
}

class ChoixTuile {
    -id : size_t
    -tuilesDisponibles : vector<TuileCite*>
    +ChoixTuile(id : size_t)
    +ajouterTuile(tuile : TuileCite*) : void
    +ajouterPile(tuiles : vector<TuileCite*>) : void
    +choisirTuile(joueur : Joueur*, index : size_t) : TuileCite*

```

```
+remettreTuile(tuile : TuileCite*, index : size_t) : void
+calculerCout(index : size_t) : size_t
+getTuilesDisponibles() : const vector&
+getNombreTuiles() : size_t
}

class TableauScore {
    -scores : vector<pair<Joueur*, int>>
    -variantesActives : vector<Variante>
    +ajouterJoueur(Joueur*)
    +calculerScores()
    +afficherScores()
    +afficherDetailsScore(joueur : Joueur)
    +calculerScoreIA(joueur : Joueur, diff : NiveauDifficulte) : int
    +calculerDetails(joueur : Joueur) : map<Couleur, ScoreDetail>
}

abstract class CalculScoreBase {
    +calculerScore(joueur : Joueur) : int
}

class CalculScoreRecouvrement
class CalculScorePlaces
class CalculScoreMultiplicateurs

class TuileCite {
    -id : size_t
    -hauteur : unsigned int
    -hexagones : array<HexagoneConstruction*, 3>
    -proprietaire : bool
    +TuileCite(...)
    +rotationHoraire()
    +setHauteur(h : int)
    +contientCarriere() : bool
    +contientPlace() : bool
    +getNombreQuartiers() : int
    +getCouleursQuartiers() : vector<Couleur>
    +clone() : TuileCite*
}

class TuileDepart {
    -id : size_t
    -hexagones : array<HexagoneConstruction*, 4>
    +TuileDepart(...)
```

```
+getHexagone(index : size_t) : HexagoneConstruction*
}
```

```
abstract class HexagoneConstruction {
  -id : size_t
  +clone() : HexagoneConstruction*
}
```

```
class Quartier {
  -type : Type*
  +getType() : Type
}
```

```
class Place {
  -type : Type*
  -nbEtoile : Etoile
  +getType() : Type
  +getNbEtoile() : Etoile
}
```

```
class Carriere {
  -coul : Couleur
}
```

```
class Type {
  -nom : string
  -couleur : Couleur
  -conditions : string
  +{static} HABITATION : Type
  +{static} MARCHE : Type
  +{static} CASERNE : Type
  +{static} TEMPLE : Type
  +{static} JARDIN : Type
}
```

' Relations

```
Partie "1" *-- "1..4" Joueur : contient >
Partie "1" *-- "1" Pioche : contient >
Partie "1" *-- "1" ChoixTuile : gère >
Partie "1" *-- "*" Variante : possède >
```

```
Joueur "1" *-- "1" Cite : possède >
Joueur ..> TableauScore : utilise >
```

Pioche "1" o-- "*" TuileCite : contient (Piles) >

ChoixTuile "1" o-- "*" TuileCite : expose >

Cite "1" *-- "*" TuileCite : stocke (posées) >

TuileCite "1" *-- "3" HexagoneConstruction : compose >

TuileDepart "1" *-- "4" HexagoneConstruction : compose >

HexagoneConstruction <|-- Quartier

HexagoneConstruction <|-- Place

HexagoneConstruction <|-- Carriere

Quartier --> Type

Place --> Type

TableauScore --|> CalculScoreRecouvrement

TableauScore --|> CalculScorePlaces

TableauScore --|> CalculScoreMultiplicateurs

TableauScore --|> CalculScoreBase

CalculScoreRecouvrement --|> CalculScoreBase

CalculScorePlaces --|> CalculScoreBase

CalculScoreMultiplicateurs --|> CalculScoreBase

@enduml